Informe Técnico: Desarrollo e Integración de Módulos de Software y Módulos Integrados (saludPorTuVida) - GA8-220501096-AA1-EV01- GA8-220501096-AA1-EV02

Sebastián Rosas Noguera

Juan Esteban Gallego Loaiza

Paola Andrea Corredor Marín

Servicio Nacional de Aprendizaje (SENA)

Análisis Y Desarrollo De Software

3070200

Yerlys Jesús Sánchez Pino

18 marzo 2026

1. INTRODUCCIÓN Y ALCANCE

El presente documento detalla el proceso de codificación, integración y pruebas de los módulos del backend del sistema **"SaludPortuVida"** (Subsistema Farmacia). Esta solución es una **API RESTful** desarrollada en Java diseñada para la gestión de inventario farmacéutico, control de acceso de usuarios y análisis predictivo de stock para la toma de decisiones.

2. ARQUITECTURA Y TECNOLOGÍAS (Requerimiento EV01)

2.1. Metodología y Patrones

Se ha utilizado una arquitectura en capas (Layered Architecture) clásica en el ecosistema Spring, facilitando el mantenimiento y la escalabilidad:

- **Lenguaje:** Java 17+.
- **Framework Principal:** Spring Boot 3.2.0 (Gestión de dependencias y configuración automática).
- **Persistencia:** Spring Data JPA (Hibernate) para la interacción con la base de datos.
- **Base de Datos:** H2 Database (En memoria) para prototipado rápido y pruebas.
- **Herramientas:** Maven (Gestión de dependencias pom.xml) y Lombok (Reducción de código repetitivo)

2.2. Capas de la Aplicación

El código fuente se organiza siguiendo el patrón MVC (Modelo-Vista-Controlador), aunque actuando principalmente como API:

1. **Capa de Controladores (API):** Ubicada en com.farmacia.controller. Expone los endpoints REST (@RestController) y maneja las peticiones HTTP (JSON).

2. **Capa de Servicio (Lógica de Negocio):** Ubicada en com.farmacia.service. Contiene la lógica de validación, cálculos de stock y reglas de negocio (@Service).

3. **Capa de Acceso a Datos (Persistencia):** Ubicada en com.farmacia.repository. Interfaces que extienden de JpaRepository para operaciones CRUD automáticas.

4. **Capa de Modelo (Entidades):** Ubicada en com.farmacia.model. Mapeo objetorelacional (ORM) de las tablas de base de datos.

3. DESCRIPCIÓN DE MÓDULOS INTEGRADOS (Requerimiento EV01 y EV02)

A continuación, se documenta cada módulo codificado, sus entradas, salidas y la integración realizada.

MÓDULO 1: CONTROL DE ACCESO Y SEGURIDAD

- **Componentes integrados:**
  - src/main/java/com/farmacia/controller/FarmaciaController.java
  - src/main/java/com/farmacia/service/FarmaciaService.java
  - src/main/java/com/farmacia/repository/UsuarioRepository.java

- **Descripción:** Gestiona la identificación y registro de usuarios en el sistema.

- **Codificación:** Se implementaron endpoints POST para recibir credenciales. El servicio utiliza Optional<Usuario> para validar la existencia del usuario en la base de datos H2.

- **Datos de Entrada (JSON):**

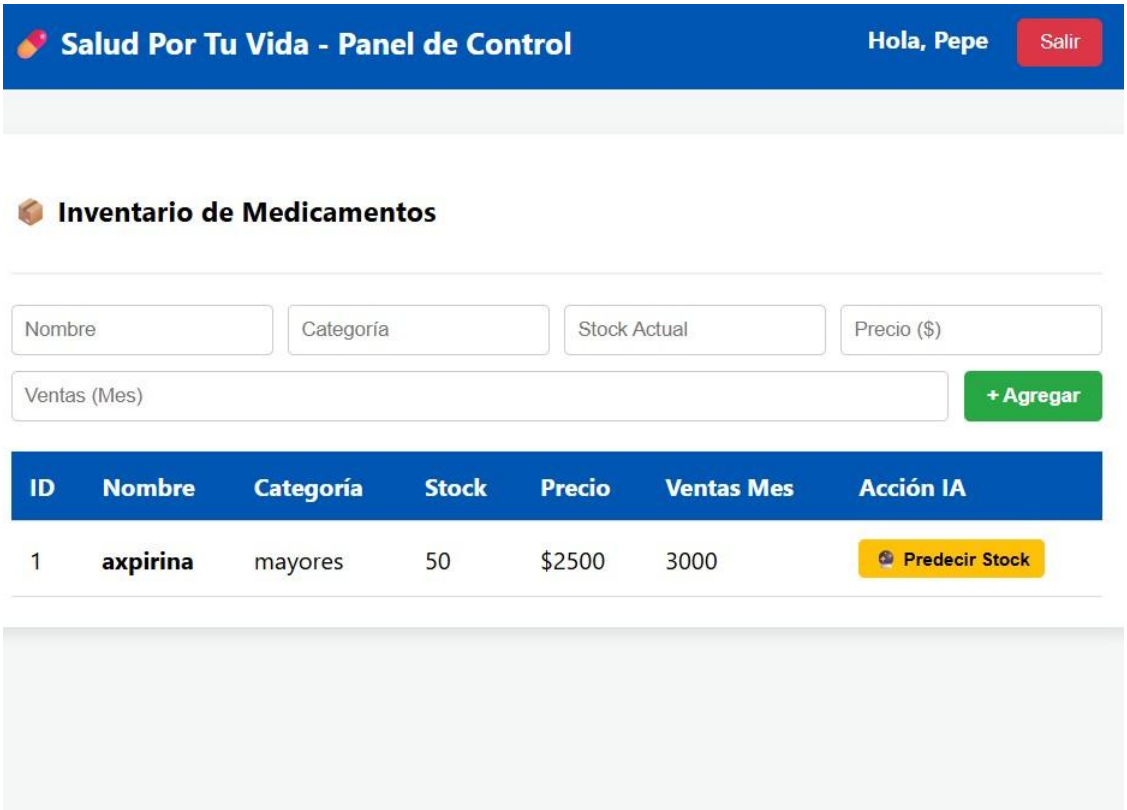
Username (String)

Password (String)

- **Datos de Salida:**

Mensaje de texto plano indicando éxito ("Bienvenido...") o fallo ("Error: Credenciales incorrectas").

MÓDULO 2: GESTIÓN DE INVENTARIO FARMACÉUTICO



Componentes src/main/java/com/farmacia/model/Medicamento.java src/main/java/com/farmacia/repository/MedicamentoRepository.java

- **Descripción:** Permite la consulta y el registro de nuevos medicamentos en el sistema.
- **Lógica Implementada:**

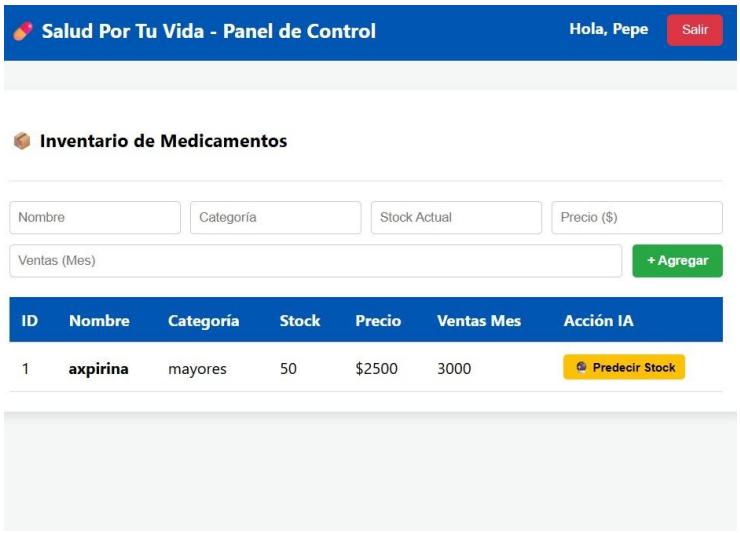
Uso de JpaRepository para realizar operaciones findAll() (Listar) y save() (Guardar) sin necesidad de escribir SQL manual.

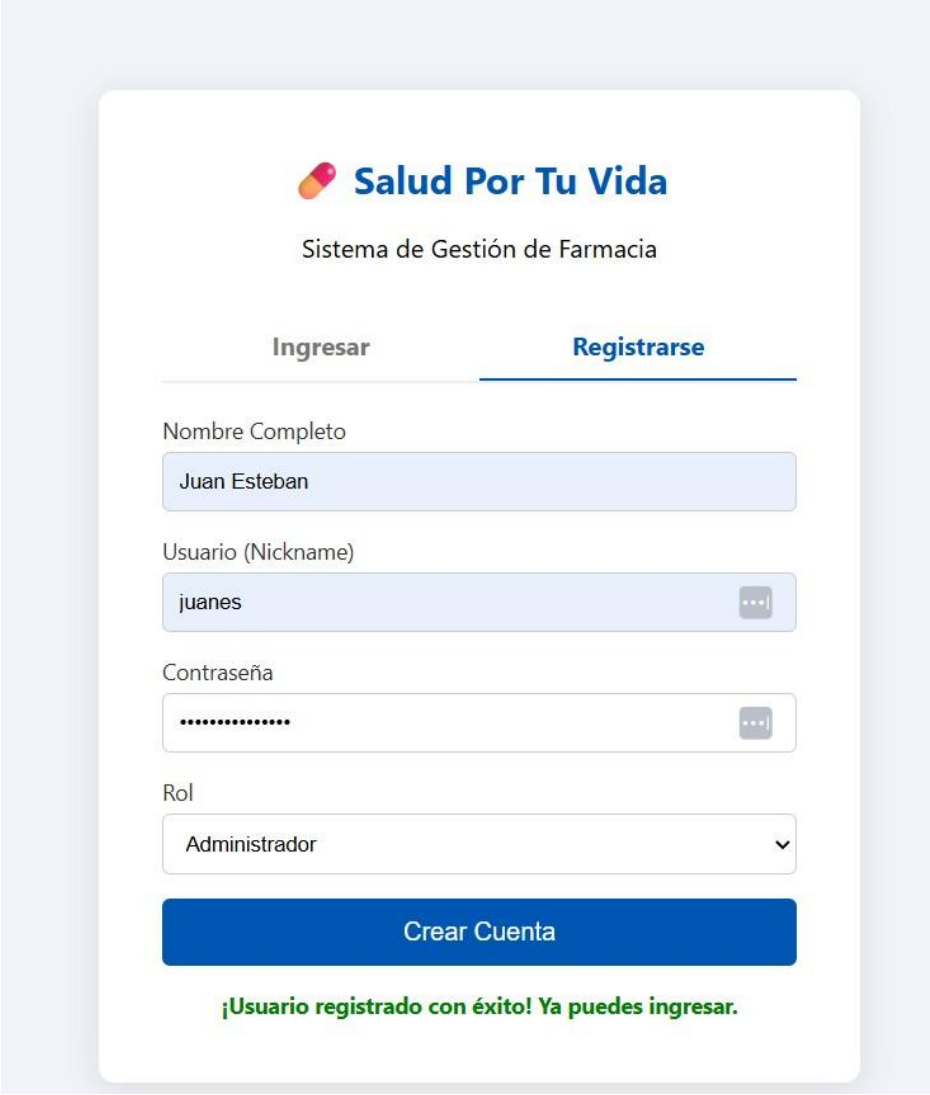
El controlador habilita CORS (@CrossOrigin(origins = "*")) permitiendo que cualquier frontend (React, Angular, etc.) consuma estos datos.

- **Entidad de Datos (Medicamento.java):**

Campos: id, nombre, categoria, stockActual, precio, ventasUltimoMes.

MÓDULO 3: SISTEMA PREDICTIVO DE STOCK (Lógica de Negocio)





- **Componentes integrados:**

src/main/java/com/farmacia/service/FarmaciaService.java (Método predecirStock) Endpoint:

GET /api/prediccion/{id}

- **Descripción:** Módulo analítico que evalúa si un medicamento está próximo a agotarse basándose en el historial de ventas.

- **Algoritmo Implementado:**

Se calcula una **Demanda Proyectada** aplicando un factor de seguridad del 20% sobre las ventas del último mes (ventas * 1.2).

Se compara el StockActual vs DemandaProyectada.

- **Datos de Entrada:** ID del medicamento.

- **Datos de Salida (String):**

Alerta Crítica: "ALERTA: Stock CRÍTICO... Comprar X unidades".

Estado Saludable: "Stock SALUDABLE...".

4. DOCUMENTACIÓN DE PRUEBAS Y RESULTADOS (Requerimiento EV02)

Se realizaron pruebas de integración (API) utilizando herramientas como Postman o el navegador, aprovechando que la base de datos es volátil (H2).

Módulo	Tipo de Prueba	Caso de Prueba	Resultado Esperado	Resultado Obtenido
Auth	API Request	POST /api/login con usuario inexistente	String: "Error: Credenciales incorrectas"	**Exitoso**
Auth	Lógica Servicio	POST /api/registro nuevo usuario	Objeto JSON del usuario creado con ID asignado	**Exitoso**
Inventario	Persistencia	GET /api/panel/medicamentos	Lista JSON vacía (inicialmente) o con datos insertados	**Exitoso**
Predicción	Lógica Negocio	GET /api/prediccion/{id} con Stock bajo	Mensaje "ALERTA: Stock CRÍTICO..."	**Exitoso**
Predicción	Manejo de Error	GET /api/prediccion/{id} ID no válido	Mensaje "Medicamento no encontrado"	

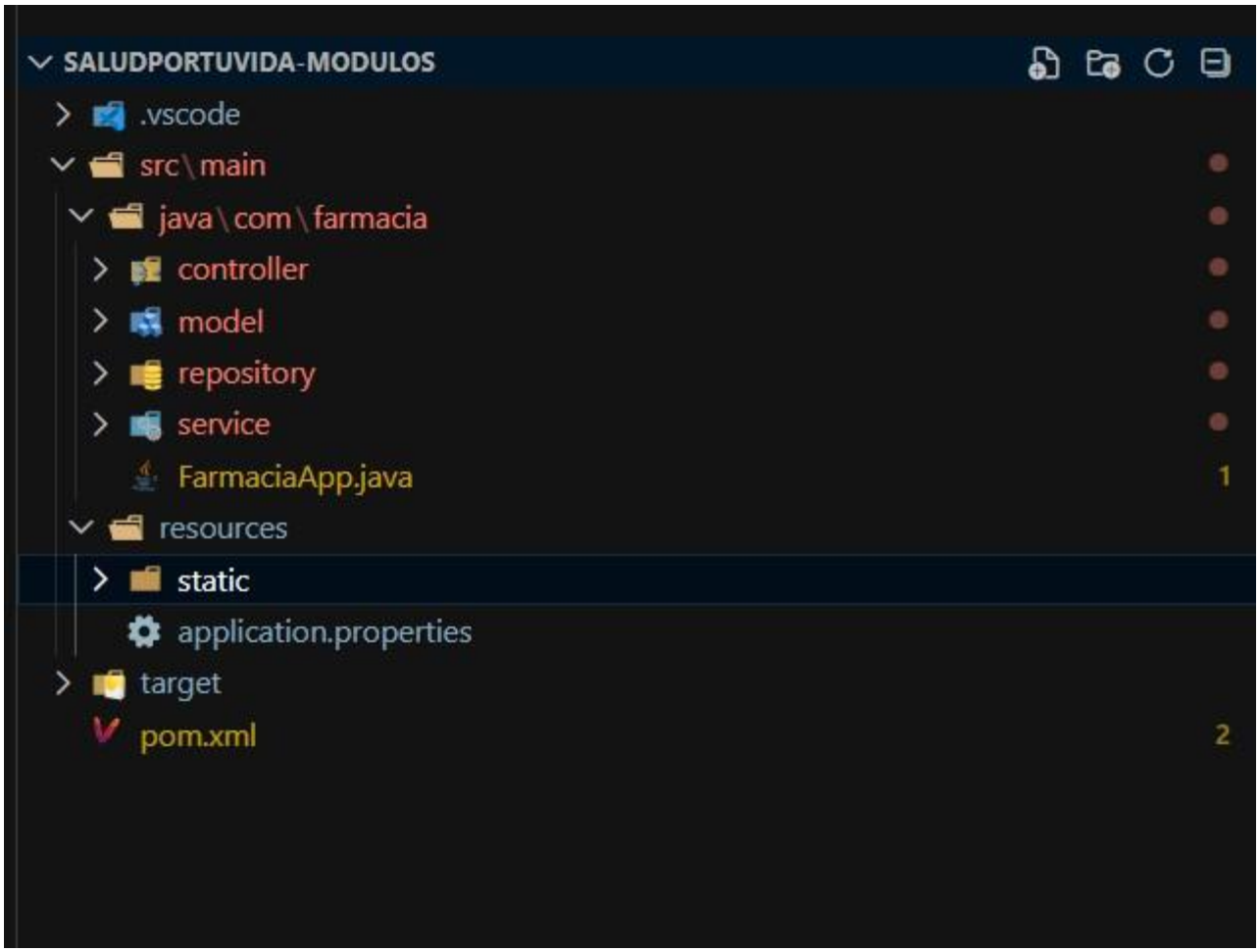
Nota: Según el archivo application.properties, se utiliza jdbc:h2:mem:farmaciadb. Esto significa que los datos se reinician cada vez que se detiene la aplicación, lo cual es ideal para el entorno de desarrollo y evaluación actual

5. REPOSITORIO Y VERSIONAMIENTO (GIT)

El código fuente respeta la estructura estándar de **Maven**.

- **URL del Repositorio:** https://github.com/juanSecuri/saludPorTuVida-app
- **Estructura del proyecto:**

```
/saludportuvida-modulos
├── pom.xml (Gestor de dependencias Maven)
├── src
│   ├── main
│   │   ├── java
│   │   │   └── com.farmacia
│   │   │       ├── FarmaciaApp.java (Main Class)
│   │   │       ├── controller (API Endpoints)
│   │   │       ├── model (Entidades JPA)
│   │   │       ├── repository (Interfaces de Datos)
│   │   │       └── service (Lógica de Negocio)
│   │   └── resources
│   │       └── application.properties (Config DB H2)
```


SALUDPORTUVIDA-MODULOS
.vscode
src\main
java\com\farmacia
controller
model
repository
service
FarmaciaApp.java
1
resources
static
application.properties
target
pom.xml
2

6. MANUAL TÉCNICO Y DE DESPLIEGUE (Requerimiento EV02)

Requisitos Previos •

Java Development Kit (JDK)

17 o superior.

- Maven 3.6+ (o Wrapper

  incluido).

- Puerto 8080 disponible.

1. **Pasos para Ejecución Local Clonar el repositorio.**

2. **Compilar :**

Ejecutar mvn clean

install en la raíz.

3. **Ejecutar la aplicación:**

```
bash

mvn spring-boot:run
```

4. **Verificación:**

5. spring-boot-starter-web: Para crear el servidor Tomcat embebido y los endpoints REST.
6. spring-boot-starter-data-jpa: Abstracción de base de datos.
7. h2: Base de datos en memoria ligera.
8. lombok: Generación automática de Getters, Setters y Constructores

Librerías Clave Utilizadas (pom.xml)

- spring-boot-starter-web: Para crear el servidor Tomcat embebido y los endpoints REST.

- spring-boot-starter-data-jpa: Abstracción de base de datos.
- h2: Base de datos en memoria ligera.
- lombok: Generación automática de Getters, Setters y Constructores.

Conclusión

Se ha logrado la codificación exitosa de los módulos backend solicitados en la evidencia. La aplicación **"SaludPortuVida (Farmacia)"** demuestra una arquitectura sólida basada en **Spring Boot**, separando claramente la exposición de datos (Controller) de la lógica de negocio compleja (Service), y preparando el terreno para una futura integración con cualquier cliente Frontend gracias a su naturaleza REST y configuración CORS.

Repositorio:

https://github.com/juanSecuri/saludPorTuVida-app

Acta de Aprobación de Requerimientos

Proyecto: Desarrollo de Aplicación Web Modular - SaludPortuVida (Subsistema Farmacia) **Fecha:** 11 de marzo de 2026 **Lugar:** Cartagena, Bolívar, Colombia

1. Objetivo del Acta

Dejar constancia de la revisión, validación y aprobación de los requerimientos funcionales y no funcionales definidos para el proyecto de desarrollo de la aplicación web modular **SaludPortuVida**, asegurando que los módulos de inventario, seguridad y análisis predictivo cumplen con las expectativas técnicas y de negocio.

2. Alcance

Esta acta cubre los requerimientos relacionados con:

- **Integración de módulos:** Login, gestión de usuarios, dashboard y notificaciones.
- **Funcionalidades de Inventario:** Operaciones CRUD y control de stock.
- **Seguridad y Control de Acceso:** Validación de credenciales y persistencia en H2.
- **Escalabilidad:** Arquitectura en capas (Controller, Service, Repository, Model).
- **Documentación:** Manuales técnicos y de usuario según estándares SENA.

3. Requerimientos Aprobados

ID	Descripción del Requerimiento
RF-01	El sistema debe permitir el inicio de sesión de usuarios (Módulo de Acceso).
RF-02	El sistema debe gestionar usuarios mediante operaciones CRUD.
RF-03	El sistema debe mostrar un panel de control con información relevante (Dashboard Farmacia).
RF-04	El sistema debe integrar un módulo de notificaciones y alertas de stock crítico.
RNF-01	El sistema debe ser modular, escalable y fácil de mantener (Arquitectura Spring Boot).
RNF-02	El sistema debe cumplir con estándares de seguridad básicos (Validación de datos y CORS).
RNF-03	El sistema debe contar con documentación técnica (JavaDoc) y manual de usuario.

4. Participantes

- **Equipo de Desarrollo:** Responsables de la implementación del backend en Java/Spring Boot.
    - Sebastián Rosas Noguera
    - Juan Esteban Gallego Loaiza
    - Paola Andrea Corredor Marín
- **Equipo de Calidad:** Responsables de validar las pruebas de integración y lógica de negocio (Stock Predictivo).
- **Cliente / Usuario Final:** Responsable de aprobar los requerimientos y flujo funcional.

5. Decisión

Tras la revisión conjunta de los módulos codificados y las pruebas de integración presentadas, **se aprueba** el conjunto de requerimientos descritos en este acta. Se autoriza formalmente al equipo de desarrollo a finalizar la documentación y continuar con la siguiente fase del ciclo de vida del proyecto.

6. Firmas de Aprobación

Nombre y Cargo	**Firma**	**Fecha**
Yerlys Jesús Sánchez Pino Gerente de TI / Instructor Líder	_____	18 de marzo de 2026